

Exercices chapitre 3-Listes

Exercice 3.1

Une suite amusante

On définit les listes $a_0, a_1, \dots$ de la façon suivante:
 $a_0 = [0]$, pour tout $n \in \mathbb{N}^*$, la liste a_n est obtenue à partir de la liste a_{n-1} de la façon suivante un 0 est remplacé par 1;0 et un 1 est remplacé par 0;1.
 Par exemple, $a_0 = [0]$, $a_1 = [1; 0]$, $a_2 = [0; 1; 1; 0]$, $a_3 = [1; 0; 0; 1; 0; 1; 1; 0]$.
 Écrire la fonction `suite n` qui renvoie la liste a_n . (`suite: int -> int list`).

Exercice 3.2

Coupure et entrelacement

1) Écrire une fonction `coupure liste n` qui renvoie un couple de deux listes, la première étant constituée des n premiers éléments de la liste passée en argument, la deuxième des éléments restant. (Si la liste contient moins de n éléments, alors on renverra le couple (`liste`, `[]`)). (`coupure: 'a list -> int -> 'a list * 'a list`).

2) Écrire la fonction `shuffle l1 l2` qui entrelace deux listes en commençant par les éléments de la première, si l'une des listes est trop courte, on termine l'entrelacement avec la fin des éléments de la liste de longueur excédentaire. (`shuffle: 'a list -> 'a list -> 'a list`).

Exercice 3.3

Sous-listes

On dit qu'une liste $[a_1, \dots, a_p]$ est une sous-liste d'une liste $[b_1, \dots, b_q]$, s'il existe des indices $i_1 < i_2 < \dots < i_p$ tels que $\forall j \in [1, p], b_{i_j} = a_j$. Autrement dit, on peut retrouver la liste a en supprimant dans b un certain nombre d'éléments de b et cela sans changer l'ordre des éléments de b .

1) Écrire la fonction `estsousliste l1 l2` de type `'a list -> 'a list -> bool` qui vérifie si $l1$ est une sous-liste de $l2$.

2) Écrire la fonction `listessedessouslistes l` de type `listessedessouslistes : 'a list -> 'a list list` qui renvoie la liste des sous-listes de l .

Exercice 3.4

Code de Gray

Un code de Gray d'ordre n est une façon d'énumérer les entiers de 0 à $2^n - 1$ telle que le passage de l'écriture binaire du k -ème nombre à celle du $k + 1$ -ème nombre se fait en changeant exactement un bit.

1) Déterminer un code de Gray d'ordre 3.

2) On va représenter un code de Gray d'ordre n , en représentant chaque nombre par la liste de ses chiffres en écriture binaire, et le code lui-même sera représenté par la liste des listes de chacun de ces nombres.
 Par exemple, la liste `[[0; 0; 0]; [0; 0; 1]; [0; 1; 1]; [0; 1; 0]; [1; 1; 0]; [1; 1; 1]; [1; 0; 1]; [1; 0; 0]]` est un code de Gray, qui correspond à l'énumération 0;1;3;2;6;7;5;4.

Montrer que si ℓ est un code de Gray d'ordre $n - 1$ pour $n \in \mathbb{N}^*$, alors en concaténant les listes de ℓ auxquelles on a ajouté un 0 en tête, et les listes de `miroir( $\ell$ )` auxquelles on a ajouté un 1 en tête, alors on obtient un code de Gray d'ordre n .

3) En déduire une fonction d'affichage d'un code de Gray d'ordre n . (`gray: int -> int list list`).

Exercice 3.5

Implémentation de la structure de liste avec un tableau

On peut définir le type `liste` en CAML de la façon suivante:

```
let taillemax = 100;;
type liste = {sommet: int; contenu: int array};;
```

On définit la liste vide par: `let vide = {sommet = -1; contenu = Array.make taillemax 0 };;` On définit alors par exemple la liste:

```
let l1 = {mutable sommet = 3; contenu = [|6;4;2;5;0|] };;
```

1) On dit que la liste l est pleine si `l.sommet = taillemax - 1`. Écrire une fonction `est_pleine l` qui teste si la liste est pleine ou non.

2) Définir les opérations élémentaires sur les listes (`est_vide`, `ajouter`, `tête`, `queue`) à l'aide de cette structure.

3) Que devient l'écriture des opérations sur les listes (`parcours`, `longueur`, `maximum`, `map`).

Exercice 3.6**Implémentation de la structure de liste par un type identifié.**

On peut définir le type liste d'entiers en CAML de la façon suivante:

```
type liste = Vide | Couple of int * liste ;;
```

Ainsi, pour définir la liste $l1 = [4;3;5]$, fait-on:

`let l1 = Couple (4, Couple (3, Couple (5, vide) ) )`. On dit que liste est un **type identifié**: Vide et Couple jouent le rôle d'identificateurs du type. On aurait pu utiliser tout autre symbole.

- 1) Définir en CAML, à l'aide de l1, la liste $l2 = [6;4;3;5]$.
- 2) Définir les opérations élémentaires sur les listes `est_vide`, `ajouter`, `tête`, `queue` à l'aide de cette structure.

Exercice 3.7**Opérations sur les polynômes**

On peut représenter un polynôme $P = \sum_{k=0}^d a_k X^k$:

- Par le tableau $t = [a_0, \dots, a_d]$ de ses coefficients.
- Par la liste ordonnée par ordre de degré décroissant des couples (i, a_i) pour $i \in I = \left\{ k \in \mathbb{N} \mid a_k \neq 0 \right\}$.

Par exemple, le polynôme $P1 = 1 + 3X + 5X^{10}$ sera représenté par le tableau $[1, 3, 0, 0, 0, 0, 0, 0, 0, 0, 5]$ ou par la liste $[(10, 5), (1, 3), (0, 1)]$.

Puisque l'on manipule ici des polynômes à coefficients entiers, on utilise le type défini par:

```
type listep = int * int list.
```

- 1) Écrire une fonction `tableau_vers_liste tp` qui renvoie la liste `lp` associée au tableau `tp`.
`tableau_vers_liste: int array -> listep.`
- 2) Écrire une fonction `liste_vers_tableau lp` qui renvoie le tableau `tp` associé à la liste `lp`.
`liste_vers_tableau: listep -> int array.`
- 3) Écrire les fonctions `add: listep -> listep -> listep` (Additions de deux polynômes), `prods: listep -> int -> listep` (produit d'un polynôme par un scalaire), `prodm` (Produit par un monome de la forme αX^j), `prodp` (produit de polynômes), `deriv`, `division` (division par un polynôme de coefficient dominant égal à 1) associées à l'aide des opérations usuelles sur les listes. Calculer leur complexité.
- 4) Comparer ces deux représentations. Avantages et inconvénients ?

Exercice 3.8**Évaluation d'une expression arithmétique**

On cherche à représenter des expressions arithmétiques comme: $(3 * 2) + ((1 + 6) * (2 - 1))$.

On va représenter une telle expression par une liste dont la tête sera l'opérateur correspondant à la dernière opération à effectuer la queue contiendra dans cet ordre: l'expression correspondant au nombre de gauche puis l'expression correspondant au nombre de droite dans la dernière opération à effectuer.

On définit donc les types suivants:

```
type operation = Plus | Foix | Moins;;
type algebre = Nombre of int | Op of operation;;
```

L'exemple, ci-dessus sera alors représenté par la liste déclarée par:

```
let essai2 = [Op Plus; Op Foix; Nombre 3; Nombre 2;
Op Foix; Op Plus; Nombre 1; Nombre 6; Op Moins; Nombre 2; Nombre 1];;
```

Écrire une fonction `evaluate: algebre list -> int` qui permet d'évaluer une telle expression. (Cela suppose implicitement que l'écriture d'une expression sous forme d'une telle liste est unique).

Exercice 3.9**Base Fibonacci**

On appelle $(F_n)_{n \in \mathbb{N}}$ la suite de fibonacci. On rappelle que tout élément de $\mathbb{N}^*$ admet une unique décomposition comme somme de nombres de Fibonacci non consécutifs. Par exemple, $131 = 8 + 34 + 89$.

1) Écrire la fonction `listefibo x` qui renvoie la liste triée par ordre décroissant de tous les nombres de Fibonacci inférieurs ou égaux à l'entier naturel x . `listefibo: int -> int list`.
Par exemple, `listefibo 125` renverra `[89; 55; 34; 21; 13; 8; 5; 3; 2; 1; 1; 0]`.

2) Écrire la fonction `decomposition x` qui renvoie la liste triée par ordre croissant des nombres de Fibonacci intervenant dans la décomposition de l'entier naturel non nul x . Par exemple, `decomposition 131` renverra `[8;34;89]`. `decomposition: int -> int list`.

3) Écrire la fonction `compose liste` de type `int list -> int` qui représente la bijection réciproque de la fonction `decompose`.

Exercice 3.10

Matrices

On manipule naturellement des matrices en `caml` à l'aide de tableaux de tableaux.

Exemples:

```
let ex1 = [| [|1;-1;2|]; [|1;1;1|]; [| -1;-1;4|] |];;
let ex2 = [| [|0;1;1|] ; [|1;1;1|]; [|1;1;1|] |];;
let ex3 = Array.make_matrix 4 3 5;;
ex3.(1).(2)<-27;;
```

1) a) Écrire les fonctions `add` (somme de deux matrices), `prods` (produit d'une matrice par un scalaire), `prodelem` (produit d'une matrice et d'une matrice élémentaire E_{ij}), `prodm` (produit de deux matrices) à l'aide de ce type.

b) Calculer leur complexité.

2) On définit une matrice par la liste ordonnée (lexicographique) des triplets $L = \left\{ (i, j, a_{i,j}) \mid (i, j) \in I \right\}$ où $I = \left\{ (i, j) \mid a_{i,j} \neq 0 \right\}$.

a) Écrire une fonction `tableau_vers_liste ta` qui renvoie la liste `la` associée au tableau `ta`.

b) Écrire une fonction `liste_vers_tableau la` qui renvoie le tableau `ta` associé à la liste `la`.

c) Écrire les fonctions `add` (somme de deux matrices), `prods` (produit d'une matrice par un scalaire), `prodelem` (produit d'une matrice et d'une matrice élémentaire E_{ij}), `prodm` (produit de deux matrices) à l'aide des opérations usuelles sur les listes. Calculer leur complexité.

3) Comparer ces deux représentations. Avantages et inconvénients ?

Exercice 3.11

Tri d'une liste

On considère un type liste d'entiers. On veut écrire une fonction `tri (l: 'a list): 'a list` qui renvoie une liste obtenue à partir de `l` en ordonnant les éléments de `l` par ordre croissant.

1) Écrire une fonction `insere (x:'a) (l: 'a list): 'a list` qui insère l'entier `x` dans la liste triée `l` de sorte que la liste obtenue soit encore triée.

2) On réalise le tri par insertion d'une liste quelconque `l` de la façon suivante:

1. Si `l` est vide: on ne fait rien.

2. Si `l` est non vide: on trie la queue de `l` puis on insère à la bonne place la tête de `l`

Donner la définition inductive de cette fonction de tri.

En déduire une écriture de la fonction `tri_insertion l` correspondante.

Évaluer sa complexité. (Signature: `tri_insertion: 'a list -> 'a list`).

3) On réalise le partage d'une liste `l` en deux listes `l1` et `l2` de la façon suivante:

. Si `l` est vide: le résultat est constitué de deux listes vides.

. Si `l` a un seul élément: le résultat est constitué de la liste `l` et de la liste vide.

. Si `l` a au moins deux éléments: on partage la liste `l` privée des deux premiers éléments en deux listes `l1` et `l2` puis on ajoute le premier élément de `l` à `l1` et le deuxième élément de `l` à `l2`.

Donner la définition inductive de cette fonction de partage.

En déduire une écriture de la fonction de signature `diviser: 'a list -> 'a list * 'a list` correspondante.

Évaluer sa complexité.

4) On veut réaliser la fusion de deux listes triées `l1` et `l2` en une liste triée `l`. Énoncer un algorithme comme dans les questions précédentes.

Donner la définition inductive de cette fonction de fusion.

En déduire une écriture de la fonction `fusion l1 l2` correspondante. Évaluer sa complexité.

(signature: `fusion: 'a list -> 'a list -> 'a list`).

5) En déduire un algorithme de tri par fusion d'une liste `l`. Écrire la fonction `tri_fusion l` correspondante. Évaluer sa complexité. Commentaires ? (signature: `trifusion: 'a list -> 'a list`).

6) Faire la même chose avec le quicksort sur les listes. Évaluer en particulier la complexité moyenne du quicksort.

(signature: `quicksort: 'a list -> 'a list`).

Exercice 3.12**Décomposition en sous-séquences monotones maximales**

Écrire une fonction `decompose liste` qui renvoie la décomposition d'une liste de sous-séquences monotones de longueur maximale.

Par exemple, `decompose [1;5;8;8;2;3;4;3;2;2;1;0;1;5;6]` doit renvoyer

`[[1; 5; 8; 8]; [2; 3; 4]; [3; 2; 2; 1; 0]; [1; 5; 6]].`

`decompose : int list -> int list list.`

Exercice 3.13**Tranche maximale d'une liste d'entiers**

On appelle une tranche d'une liste l , une liste obtenue en sélectionnant un certain nombre de cases consécutives de la liste l .

1) Écrire une fonction de complexité linéaire qui renvoie la somme maximale des tranches de l .

`tranche_max: int list -> int.`

(indication: on pourra commencer par écrire une fonction qui calcule deux choses: 1) la valeur maximale des sommes de tranches de l qui commencent en début de liste, et 2) la valeur maximale des sommes de tranches de l .)

2) Adapter le principe précédent, pour écrire une fonction `tranche_max_elements` qui renvoie la somme maximale des tranches d'une liste et une tranche de somme maximale.

`tranche_max_elements: int list -> int * int list.`

Par exemple, `tranche_max_elements [-2;1;3;5;-1;10; 1; -3;-2; -6;-6]` renverra `(19, [1; 3; 5; -1; 10; 1])`

Exercice 3.14**Un jeu de solitaire**

Notre jeu de solitaire se présente sous la forme d'une règle graduée de n cases numérotées de 1 à n . Les règles du jeu sont les suivantes:

- Une case donnée contient 0 ou 1 jeton.
- On ne peut modifier qu'une seule case à la fois.
- On peut à tout moment remplir la case 1 si elle est vide, ou vider la case 1 si elle est pleine.
- On peut modifier le contenu de la case $k \in [2, n]$ (c'est-à-dire la vider si elle est pleine, la remplir si elle est vide), si et seulement si la case $k - 1$ est occupée et toutes les cases $1, \dots, k - 2$ sont vides.

On code une séquence de mouvement à effectuer, par liste des cases dont on modifie le contenu. Par exemple, on vérifiera que la liste `[1; 2; 1; 3; 1; 2; 1; 4; 1; 2]` code une suite de déplacement permettant de remplir un solitaire de 4 cases.

1) Écrire les fonctions `remplir n` et `vider n` qui permettent de remplir ou de vider un solitaire de n cases.

2) Estimer la complexité de chacune de ces fonctions.

`remplir: int -> int list` `vider: int -> int list.`

Exercice 3.15 Énumération des parties d'un ensemble, des applications d'un ensemble vers un autre

On représente un ensemble fini $E = \{e_1, \dots, e_n\}$ par la liste de ses éléments deux à deux distincts $[e_1, \dots, e_n]$. Une collection de parties de E est donc une liste de liste d'éléments de E .

On représente une fonction f entre deux ensembles E_1 et E_2 par la liste des couples $(x, f(x))$ pour $x \in E$. Par exemple, la liste `[(a, 1), (b, 3), (c, 3)]` représente la fonction f de l'ensemble à 3 éléments a, b et c vers $[1, 4]$, telle que $f(a) = 1$ et $f(b) = f(c) = 3$.

Signature des fonctions à écrire:

`ajout: 'a -> 'a list list -> 'a list list.`

`parties: 'a list -> 'a list list.`

`popieme: 'a list -> int -> 'a * 'a list.`

`permutation: 'a list -> 'a list list.`

`surjection: 'a list -> 'b list -> ('a * 'b) list list.`

1) Écrire une fonction `ajout x collection` qui ajoute l'élément x à chacune des parties de la collection `collection`. (On suppose que les parties de la collection ne contiennent pas initialement l'élément x).

2) En déduire une fonction `parties ensemble` qui renvoie une liste de liste représentant la collection de toutes les parties de E . On pourra s'appuyer sur la définition inductive suivante, si $E = E' \cup \{e\}$ et $e \notin E'$, alors $parties(E) = parties(E') \cup (ajout(x, parties(E')))$.

3) Écrire une fonction `combinaison p ensemble` qui renvoie la collection de toutes les parties à p éléments de E .

4) a) Écrire une fonction `pop_ieme i liste` qui renvoie le couple (x, lp) où x est le i ème élément de la liste `liste` et `lp` la liste obtenue en supprimant le i ème élément de `liste`.

b) Écrire une fonction `permutation ensemble` qui renvoie la collection de toutes les permutations de l'ensemble E .

5) Écrire une fonction `bijection ens1 ens2` qui renvoie la liste de toutes les bijections de l'ensemble `ens1` vers l'ensemble `ens2`.

6) Écrire une fonction `surjection ens1 ens2` qui renvoie la liste de toutes les surjections de l'ensemble `ens1` vers l'ensemble `ens2`. On pourra remarquer que si E et F sont non vides, après avoir particularisé un élément e de E et un élément f de F , on peut partitionner les surjections ϕ de E vers F telles que $\phi(e) = f$ en deux classes: la classe formée des surjections dont la restriction de ϕ à $E \setminus \{e\}$ est encore surjective, et la classe est formée des surjections telle que f n'admet qu'un seul antécédent.

Exercice 3.16

Points les plus proches

On se donne un nuage de n points (à coordonnées entières du plan). Comment déterminer en $\Theta(n \ln(n))$ opérations élémentaires, la plus courte distance entre deux points distincts de ce nuage?

Exercice 3.17

Détermination de l'enveloppe convexe

On se donne un nuage de points du plan à coordonnées entières. Les points sont représentés par le type `type point = int * int` et le nuage par le type `type nuage = point array`.

On cherche à déterminer le bord de l'enveloppe convexe de cet ensemble de points. C'est-à-dire déterminer les points qui délimitent le plus petit convexe contenant tous les points du nuage. Par exemple, pour le nuage défini par `let ex_nuage = [| (0,0); (1,4); (1,8); (4,1); (4,4); (5,9); (5,6); (7,-1); (7,2); (8,5); (11,6); (13,1) |]`

Le bord de l'enveloppe convexe est constituée des points dont les indices dans `ex_nuage` sont ceux de la liste `[7; 11; 10; 5; 2; 0]`.

On suppose que les points du nuage sont classés par ordre d'abscisse croissante.

1) a) Justifier que le point minimal du nuage pour l'ordre lexicographique (*ordonnee, abscisse*) est dans le bord de l'enveloppe convexe.

b) Écrire la fonction `point_init` de type `nuage -> int` qui renvoie l'indice du point minimal du nuage pour l'ordre lexicographique (*ordonnee, abscisse*).

2) On définit l'orientation de trois points du plan A_i, A_j et A_k par:

- Si $A_i A_j A_k$ est un triangle non plat direct, alors l'orientation de ces trois points est 1.
- Si $A_i A_j A_k$ est un triangle non plat indirect, alors l'orientation de ces trois points est -1 .
- Si deux des trois points sont égaux, alors l'orientation est égale à 0.

Écrire la fonction `orientation` de type `nuage -> int -> int -> int -> int` qui prend en argument un nuage de points et trois indices du tableau `nuage` et renvoie l'orientation des points correspondant à ces indices.

3) Lorsque i est un indice du tableau `nuage`, on note A_i le point du plan. On note $E = \{A_0, \dots, A_{n-1}\}$ les points du nuage.

a) Soit i un indice du tableau `nuage` qui correspond à un point du bord de l'enveloppe convexe du nuage. Montrer que l'on définit une relation d'ordre totale sur $E \setminus \{A_i\}$ en posant:

$$A_j \preceq_i A_k \iff \text{orientation}(A_i, A_j, A_k) \leq 0.$$

b) Écrire une fonction `point_max` de type `nuage -> int -> int` qui renvoie, pour un nuage donné et un indice i correspondant à un point A_i du bord de l'enveloppe convexe du nuage, l'indice du point du nuage maximal pour la relation $\preceq_i$.

c) Interpréter géométriquement le point renvoyé par la fonction `point_max`.

4) a) En déduire une fonction `jarvis` de type `nuage -> int list` qui renvoie la liste des indices des points du bord de l'enveloppe convexe du nuage.

b) Estimer la complexité de la fonction `jarvis` en fonction du nombre m de points du bord de l'enveloppe convexe et du nombre n de points du nuage.

Exercice 3.18

Listes paresseuses

On se propose d'implémenter, en OCaml, une bibliothèque de calcul paresseux. Celle-ci expose un type paramétré de suspensions `'a susp` et des fonctions de types suivants :

`susp` : `(unit -> 'a) -> 'a susp`

`force` : `'a susp -> 'a`. Une suspension (de type `'a susp` contient une fonction permettant de calculer une valeur de type `'a`. Elle peut être construite facilement grâce à la fonction `susp`. Le calcul n'est effectué que lorsque l'utilisateur de la bibliothèque le demande via la fonction `force`. Cette dernière fonction vérifie si le calcul a déjà été effectué : si tel est le cas, elle en renvoie le résultat précalculé. Sinon, elle lance le calcul, stocke le résultat pour de futurs appels, et elle le renvoie.

En voici, une implémentation CAML possible:

```
type 'a susp_state = Computed of 'a | Suspended of (unit-> 'a);;
type 'a susp = 'a susp_state ref;;

let susp f = ref (Suspended f);;

let force f =
  match !f with
  | Computed bidule -> bidule
  | Suspended g -> let bidule = g () in
    f := Computed bidule; bidule;;
```

On peut partir de là définir de la façon suivante une structure de liste paresseuse:

```
type 'a maillon = Rien | Cons of 'a * 'a slist
and 'a slist = 'a maillon susp;;
```

La liste vide peut alors être codée par

```
let liste_vide = susp (fun () -> Rien);;
```

1) Écrire une fonction `scons` : `'a -> 'a slist -> 'a slist` qui ajoute un élément en tête d'une liste paresseuse.

2) Écrire deux fonctions `shd` : `'a slist -> 'a` et `stl` : `'a slist -> 'a slist` qui prennent une liste paresseuse non vide en paramètre, et qui renvoient respectivement son premier élément et la liste paresseuse des autres éléments.

3) Écrire une suite d'instruction permettant de définir la liste paresseuse `[13; 12; 10]`.

4) Écrire une fonction `affiche` : `int slist -> int -> unit` telle que `affiche l n` affiche les n premiers éléments d'une liste paresseuse l .

5) Définir les listes `liste_carres` et `liste_cubes` paresseuses "infinies" (et c'est là que c'est cool) de tous les carrés d'entiers et de tous les cubes d'entiers.

6) Écrire une fonction `sappend` : `'a slist -> 'a slist -> 'a slist` qui concatène de manière paresseuse deux listes paresseuses. Cette fonction devra s'exécuter en temps constant. (Vérifier en concaténant `liste_carres` et `liste_cubes`).